

Diabetic Retinopathy Pipeline

Implementation & Session Documentation

Generated: 2026-09-16 17:48:57

Machine: DESKTOP-39I5E6Q (Windows 11)

Project root: C:\Users\HP\Downloads\dr-project-python-fixed\dr-project-python-master

This document records the work done to take the rebuilt dr/data/ package from an untested checkout to a running, resumable, full-scale training pipeline on this machine - every real bug found along the way, the checkpoint/resume architecture added because this machine cannot stay powered on for the full multi-day run, the CSV export additions, and a live snapshot of where the pipeline stands as of the timestamp above. Re-run this script any time to refresh that snapshot.

1. Environment Setup

The project ships requirements.txt pinned to torch>=2.2 with no Python version constraint; this machine's only available interpreter is Python 3.14, which is new enough that the default PyTorch CUDA wheel index (cu121) has no build for it at all.

- Created .venv on the system Python 3.14 interpreter.
- Installed PyTorch 2.11.0 + torchvision from the cu128 wheel index (download.pytorch.org/whl/cu128) - the cu121/cu124 indices either lack a 3.14 build or lack Blackwell (RTX 50-series) kernel support.
- Installed the remainder of requirements.txt (timm, opencv-python-headless, scikit-learn/-image, pandas, matplotlib, onnx/onnxruntime, kaggle) - all resolved cleanly for 3.14.
- Placed the user-supplied Kaggle API credentials at %USERPROFILE%\kaggle\kaggle.json so scripts/11_download_lesions.py could fetch the real IDRiD and DDR lesion-annotation corpora.

2. GPU Functionality

Verified end-to-end before committing any real compute time: package installation alone does not prove the CUDA kernels actually match this GPU's architecture.

PyTorch build: 2.11.0+cu128

CUDA available: True

Device: NVIDIA GeForce RTX 5060

VRAM: 8.5 GB

Compute capability: sm_120

Verification performed: allocated a tensor on the CUDA device and ran a real matrix multiply on it (not just an availability check) - confirmed the cu128 build's sm_120 kernels execute correctly on this Blackwell-architecture card, which is not guaranteed by an older CUDA build simply importing without error.

Constraint this GPU imposes on the rest of the run: the project's own full-schedule script (`run_gpu_full.sh`) assumes ≥ 16 GB VRAM at batch-size 32 for a ViT-Large backbone across a 448px global view plus 6 local crops. This card has 8GB, so the training stage uses `--batch-size 4 --grad-accum 8 --grad-checkpoint` (same effective batch of 32, same total compute, more but smaller steps) as the starting point - see `resume_pipeline.ps1`.

Observed throughput on this GPU/CPU combination so far:

- Preprocessing (CPU-bound, 24 worker processes): ~ 1.7 - 2.0 images/sec sustained on real 1024px EyePACS images.
- Lesion-expert pretraining (GPU-bound, small model): ~ 8 - 9 images/sec at 448px on the real IDRiD/DDR crop cache.
- Full staged training throughput has not been measured yet - preprocessing must finish first, since training reads from its cache.

3. Real Bugs Found and Fixed

This code had clearly never been run end-to-end against real, full-scale data before - every one of these surfaced only once actual EyePACS/IDRiD/DDR data and a real training step were exercised, not from the project's own synthetic self-test.

3.1 `src/dr/data/eyepacs.py`

Problem: `build_manifest()` only searched a few hardcoded subfolders for images and never found this machine's actual layout (`resized_train/resized_train/`); it silently found 0 of the 35,126 images.

Fix: Added the actual nested layout (and its `_cropped` variant) to the search list.

3.2 `src/dr/data/lesion_datasets.py`

Problem: Folder discovery picked the FIRST matching image/mask folder found anywhere in the tree. For IDRiD (three sections, each with its own 'Original Images' folder) this paired images from the wrong section against the mask folder; for DDR (train/test/valid splits) it paired one split's images against another split's masks. Both cases produced 0 (image, mask) pairs.

Fix: Rewrote `discovery` to merge every matching folder by filename stem instead of picking one arbitrarily. Now correctly finds all 81 IDRiD and 757 DDR annotated images.

3.3 `scripts/08_selftest.py`

Problem: The crop-centre assertion expected normalised 0-1 coordinates, but `generate_lesion_crops()` returns pixel coordinates in the cache's own pixel grid (confirmed against how `02_preprocess.py` and `CachedEyePACS` actually consume the values) - a self-test bug, not a production bug.

Fix: Fixed the assertion to check against the `cache_size` bound instead of `[0,1]`.

3.4 `src/dr/modules/a4_backbone_lora.py`

Problem: When `require_retfound=False` and no checkpoint could be resolved at all, `load_retfound_weights()` returned a partial report dict; the status-string builder unconditionally read fields (`matched_tensors`, etc.) that only exist on a successful load, crashing with `KeyError`.

Fix: Branch on whether the load actually succeeded before building the status string; fall back to a clear

'not loaded' message otherwise.

3.5 scripts/10_lesion_pretrain.py

Problem: Called `train_ds.lesion_prevalence()` as a method; it is a `@property` already returning a dict, so calling it raised `TypeError`. The print statement right behind it then zipped a 6-name tuple against the resulting 4-key dict, which would have crashed again on the next attempt. Separately, the training loop read batch keys 'lesion_masks'/'presence' that the dataset never produces (it yields 'masks'/'valid', with no separate presence label at all).

Fix: Removed the erroneous call, fixed the print to iterate the dict directly, and derived presence from the masks (per-channel: any positive pixel in the crop).

3.6 src/dr/modules/external_lesion_batches.py and scripts/05_predict.py

Problem: Both independently treated `generate_lesion_crops()`'s pixel-space centres as 0-1 fractions and multiplied by image height again, pushing every centre out of range. Since both clip bounds are equal in that regime, every crop for a given image collapsed onto the same fixed corner regardless of where the annotated lesion actually was - this fed both the real-mask auxiliary training path and the Objective 4 XAI evidence path.

Fix: Removed the erroneous re-multiplication in both files.

3.7 scripts/03_train.py

Problem: The training loop read a batch key called 'index' that `CachedEyePACS` never produces (it's called 'cache_row') - would have crashed on the first training step.

Fix: Changed the read to 'cache_row'.

3.8 scripts/03_train.py

Problem: `HardExampleState.empty(len(train_ds.meta))` sized the hard-example-mining arrays to the TRAIN-SPLIT count (~28,102), but they are indexed by `cache_row`, a global row index assigned before the train/val/test split (so it ranges over the whole ~35,126-row cache). Any sample whose `cache_row` fell outside the narrower train-count range would index out of bounds.

Fix: Sized the state to `len(train_ds.meta_full)` instead, matching how `CachedEyePACS` already sizes its own internal hardness buffer for the same reason.

4. Checkpoint / Resume Architecture

This machine cannot stay powered on for the multi-hour preprocessing pass or the multi-day training run, so neither script could originally survive an interruption - a shutdown mid-run meant restarting from image 1 or epoch 1. Both now checkpoint and auto-resume.

4.1 scripts/02_preprocess.py

- Writes one row to `preprocess_progress.csv` per successfully processed image (flushed + fsynced every 200 images), and a separate `preprocess_failed.json` for permanently-unreadable/errored images so they are excluded from the dataset without being retried forever on every resume.
- On startup: if the cache files already exist at the exact shape this invocation implies, opens them in `r+` mode (instead of overwriting) and skips every already-completed or already-failed index.

- meta.csv (the file training actually reads) is written to a .tmp path and atomically renamed into place, so a crash mid-write can never leave a corrupt/partial meta.csv.
- Verified live: stopped the real run after ~400 images, restarted the exact same command, and confirmed it printed '400/35126 already cached, continuing with the remainder' instead of reprocessing from scratch.

4.2 scripts/03_train.py

- Saves a full checkpoint.pt after EVERY completed epoch (not only new-best epochs): model weights, optimizer state, LR-scheduler state, EMA shadow weights, the hard-example-mining state, training history, best/best_epoch/stale, and the torch/CUDA/numpy RNG states - written to a .tmp file and atomically renamed, so a shutdown mid-save cannot corrupt the one checkpoint a resume depends on.
- On restart: reconstructs the LoRA adapter architecture deterministically from the saved per-block ranks (bypassing the RNG-sensitive GLA-LoRA calibration pass entirely, since the LoRA layers are real injected modules that model.load_state_dict() needs to already exist), restores every piece of state above, skips any fully-completed stage, and resumes the partially-completed stage at the next epoch with its original optimizer/scheduler state.
- Worst-case loss from an unplanned shutdown: the single epoch in progress, never more.

4.3 scripts/10_lesion_pretrain.py

Given the same per-epoch checkpoint treatment for consistency, though its crop-cache reuse already made a full re-run cheap.

4.4 resume_pipeline.ps1

A single launcher at the project root chaining self-test -> preprocessing -> lesion pretraining -> staged training -> the full evaluation/reporting pipeline. Safe to re-run after any restart - each stage auto-detects and continues from its own checkpoint. Usage:

```
powershell -ExecutionPolicy Bypass -File resume_pipeline.ps1
```

Caveat: once training has written its first checkpoint, the arguments inside this script (particularly --batch-size/--grad-accum) should not change - a resumed run reuses the saved LR-schedule state, which a change would throw off. The final evaluation/report stages are not individually checkpointed (only the two multi-hour stages are); if one of those is interrupted, only that one step needs re-running.

5. CSV Export Additions

A shared helper, src/dr/csv_export.py (save_csv_alongside), writes a best-effort tabular CSV next to whichever JSON a script already produces, wherever the data has one obvious shape: a list of flat dicts becomes one row per entry; a dict of dicts becomes one named row per key; a flat dict becomes a single summary row. Verified against each of these shapes before wiring it into every script below.

- 03_train.py: history.csv (per-epoch curve), gla_lora.csv (per-block LoRA ranks/importance)
- 10_lesion_pretrain.py: history.csv, sources.csv
- 07_compare_models.py: comparison.csv (QWK-sorted leaderboard), results.csv (every field), significance.csv
- 04_evaluate.py: evaluation.csv (long format: group/metric/value), evaluation_confusion_matrix.csv

- 06_deploy.py: distilled_metrics.csv, distilled_variants.csv
- 09_screening.py: screening.csv, screening_deployable.csv
- 12_lesion_visibility.py: lesion_visibility.csv
- 13_linear_probe.py: linear_probe_<tag>.csv
- 14/15/18/20/22_*.py: one CSV per objective evidence-pack JSON
- 16_ordinal_calibration.py: ordinal_calibration_results.csv
- 19_objective3_validation.py: objective3_validation_arms.csv
- 21_xai_evaluation.py: xai_evaluation_per_image.csv, xai_evaluation_metrics.csv
- 23_objectives_summary.py: objectives_consolidated.csv

6. Current Pipeline Status (live at generation time)

6.1 Preprocessing

Images cached so far: 35,126 / 35,126

Last log lines:

```
[preproc] 14901/35126  1.9 img/s (this run)  eta 180.3 min
[preproc] 15401/35126  1.9 img/s (this run)  eta 177.1 min
[preproc] 15901/35126  1.9 img/s (this run)  eta 172.4 min
[preproc] 16401/35126  1.9 img/s (this run)  eta 167.8 min
```

6.2 Lesion-Expert Pretraining

COMPLETE - 8 epochs run, best validation Dice 0.1401. Checkpoint: outputs/lesion_pretrain/lesion_encoder.pt

6.3 Staged Training

Not started - waiting on preprocessing to finish.

6.4 Disk / Environment

Free disk space: 726.3 GB

GPU: NVIDIA GeForce RTX 5060 (8.5 GB VRAM)

7. Next Steps

- Let preprocessing finish (full 35,126-image, 1024px cache).
- Start scripts/03_train.py with --lesion-encoder pointed at the completed lesion_encoder.pt (already wired into resume_pipeline.ps1), starting at --batch-size 4 --grad-accum 8 --grad-checkpoint for the 8GB card.
- If training OOMs before its first checkpoint, lower --batch-size further and raise --grad-accum to compensate - safe to do at that point since nothing has been checkpointed yet.
- Once training completes all four stages, run the full evaluation/reporting sequence (already chained in resume_pipeline.ps1) and read outputs/objectives_consolidated.json / .csv first.
- Re-run this script (generate_session_report.py) at any point to refresh this document with current

progress.